

CharLES Gap Setup Manual

Mordechai Sambol

Version 1.0, July 27, 2025

In this manual, I will explain how to use the code I wrote to set up the files needed to simulate and analyze flow over a gap using CharLES. The first section will deal with the files, how they are organized and how they should be run, and the second section will deal with the simulation they set up and what the different parameters that can be used mean.

1 Running The Setup

First, make sure that your local computer can run both Python and MATLAB scripts, as the setup requires both. After doing that, in order to allow the Python code to call MATLAB scripts, open a terminal and run the following two lines:

```
$ cd "MATLABROOT/extern/engines/python"
$ python setup.py install
```

Replace MATLABROOT with your MATLAB installation directory (e.g. /usr/local/MATLAB/R2024a). You may need to use sudo to call the installation line.

Next, in this overleaf project, under 'zip files' I have a zip titled CharLES_gap_setup_[date].zip. This file should be copied to a working directory on your **local** computer.

Unzip using

```
$ unzip CharLES\_gap\_setup\__[date].zip
```

Then remove the zip file. Run:

```
$ chmod -R 755 CharLES_gap_setup/
```

to set the editing permissions on that folder to default.

When my Python script runs, it calls a MATLAB script. The way this works is a MATLAB terminal will open in the background and run the script, and then return the result to python. This means that runtime is increased by the time it takes MATLAB to open (which can be a few seconds). If multiple runs are to be done at once, it may be better to change the Python code to interact with an open MATLAB terminal, and open one terminal at the beginning instead of opening and closing one each time. This is done by changing `eng = matlab.engine.start_matlab()` to `eng = matlab.engine.connect_matlab()`

in the python code, and running `matlab.engine.shareEngine` in the open MATLAB window (will have to be done each time a new MATLAB window is to be used).

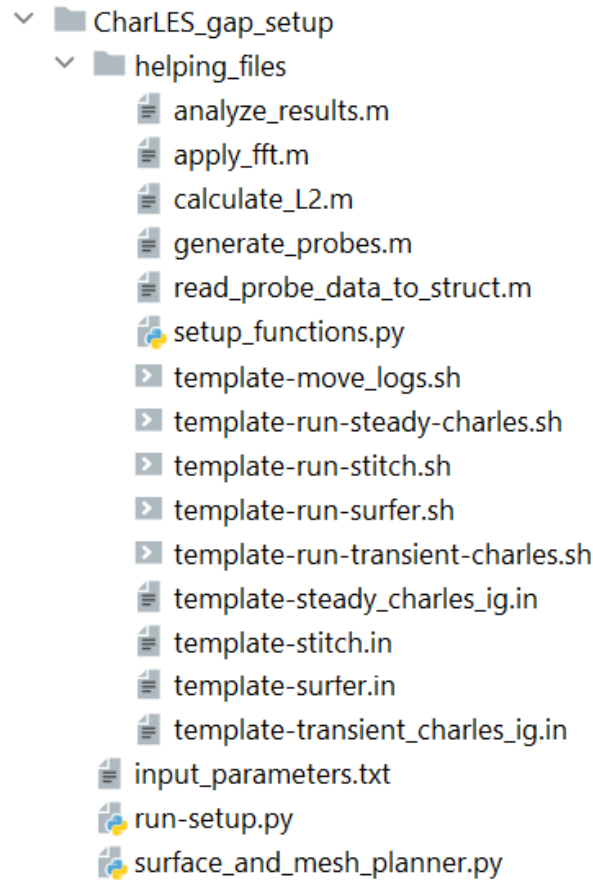


Figure 1: CharLES_gap_setup File Layout

Back to running the setup: If you return now to the directory 'CharLES_gap_setup' on your local computer, you will see that it has the layout shown in Figure 1. A quick explanation: The file 'run_setup.py' is the code that you will actually run in order to set up the files for your simulation. The file 'input_parameters.txt' contains all the run parameters. The file 'surface_and_mesh_planner.py' is meant to help you choose appropriate run parameters, by calculating and printing the length of the plate before the gap needed in order to give $\delta^* = 1$ at the entrance to the gap (for smarter choice of the length after the gap and the height of the domain), and recommending which mesh parameters will give $y^+ \approx 1$ along all the solid surfaces. The files 'template-[surfer/stitch/steady_charles_ig/transient_charles_ig].in' are templates that the python code uses in order to generate the input files

'[surfer/stitch/steady_charles_ig/transient_charles_ig].in'. These can be changed if you wish to add different images or probes to your simulation. Generally, we assume that you set them up once and then don't need to touch them again. The files 'template-run-[surfer/stitch/steady_charles_ig/transient_charles_ig].sh' are templates that the python code uses in order to generate the run files

'run-[surfer/stitch/steady_charles_ig/transient_charles_ig].sh'. These should only be changed if you change the queue you are working on, the number of nodes you are using, or other similar things. The files 'calculate_L2.m', 'generate_probes.m', and 'setup_functions.py'

contain functions that are used by 'run-setup.py', and the files 'analyze_results.m', 'apply_fft.m', and 'read_probe_data_to_struct.m' are functions used in analyzing the results of the simulation. Finally, the file 'template-move_logs.sh' is a short code copied as is to the simulation files. After running simulations, this code can be run to clean up the workspace, by moving all the log files into a designated directory.

In order to run the setup, start by opening the file 'input_parameters.txt'. An example of how this file looks can be seen in Figure 2. Change the parameter values to those you want for your simulation, then save the file. Note that each row in this file that is not a title contains a description, value, and type. The description should **not** be changed. Only change the value and if necessary the type, without changing the rest of the file layout.¹

Notice that at the end of the input file there is a section for the probe parameters, as can be seen in Figure 3. These are entered differently than the other parameters: For each collection of probes, enter a dash separated array with the number of probes that you want parallel to the surface they are on, the number of probes you want perpendicular to that surface, and the distance from that surface you wish the probes to reach. For example, the first collection of probes is the probes placed along the plate before the gap. With the present input, there are 101 probes along the x axis, 101 probes along the y axis, and the probes span the whole length of this surface, and $2 * \delta^*$ above it.

For an image of the probe placements, see Figure 4.

```
Run name, Test_run, string
##### Geometry Parameters #####
Gap length (L/delta_star), 25, double
Gap depth (D/delta_star), 10, double
Pregap slip length (L1/delta_star), 10, double
Postgap length (L3/delta_star), 40, double
Domain height (H/delta_star), 40, double
Scaling factor, 1, int
##### Mesh Parameters #####
Mesh size at far field in units of delta_star (HCP_DELTA), 1, int
Max refinement level (hat(gamma)), 5, int
Number of cells above solids with same refinement (N), 10, int
Number of cells in every following layer of refinement (NLAYERS), 10-10-10, array
Number of smoothing iterations (NSMOOTH), 10, int
##### Flow Parameters #####
Reynolds delta_star, 600, double
Mach, 3, double
Inflow speed (U_inf), 1, double
Inflow density (rho_inf), 1, double
Inflow temperature (T_inf), 1, double
gamma, 1.4, double
```

Figure 2: Input File Layout

¹To add new parameters or change the description of existing ones, add a description, value and type to the input file, then under the first function in setup_functions.py add the 'translation' between the variable description to how you want the variable to be called in the code.

```
##### Probe Parameters #####
Number of shear layer probes, 20, int
### num_of_parallel_probes - num_of_perpendicular_probes - max_distance_from_wall/delta_star ###
Pregap noslip probes, 101-101-2
Gap front wall probes, 101-1-0
Gap floor probes, 101-1-0
Gap back wall probes, 101-1-0
Postgap probes, 101-101-4
```

Figure 3: Input File Layout - cont.

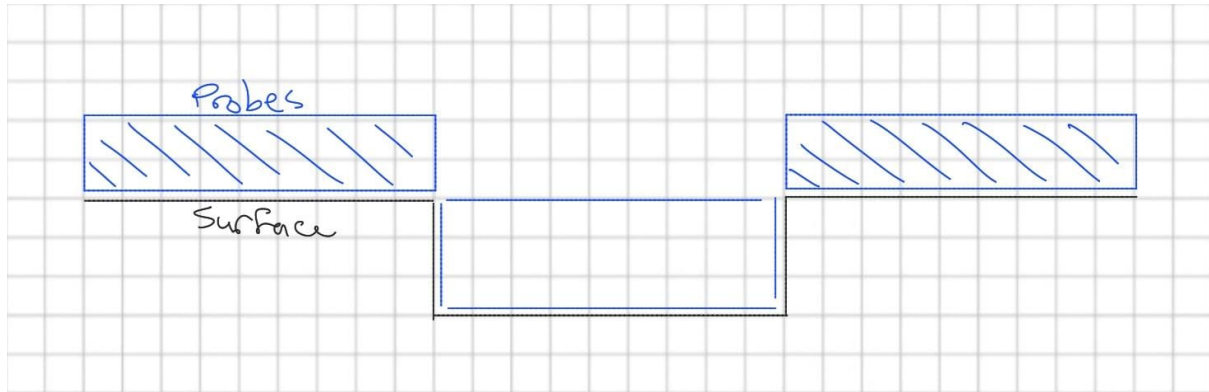


Figure 4: Probe Placements

After saving the input file, run the Python code using:

```
$ python3 run-setup.py
```

This will take a few seconds (because it needs to open and use MATLAB). When the run is complete, it should print 'Finished Simulation Setup'. You will then see that outside of the 'CharLES_gap_setup' two new directories have been created, one with a name matching the 'Run name' variable in the input file, and the other with the same name with the addition of '_analysis' at the end. the first of these directories contains all the files needed to run a simulation using CharLES, and should be moved to your working directory in zeus, using:

```
$ scp -r [Run_name] mordechais@zeus.technion.ac.il:/home/mordechais/
work/CharLES/simulations/
```

The second of these directories is meant for post-processing of the simulation results. It already contains in it MATLAB codes for reading probe results and running basic analyses (which should be changed if the required analyses change or if different probes are used).

Next to these directories (on the local computer) will appear a new file titled '[Run name]_parameters.txt'. This file is the same as 'input_parameters.txt' except with the addition of a few parameters that were calculated by the python code (like domain width and the length L2). It is meant for use by other programs, like post-processing MATLAB scripts. The new directory and file are shown in Figure 5.

Once you've moved the new directory ('Test_run' in our example) to zeus, you are ready to run your simulation. All the log files will be created under './logs/', and the other

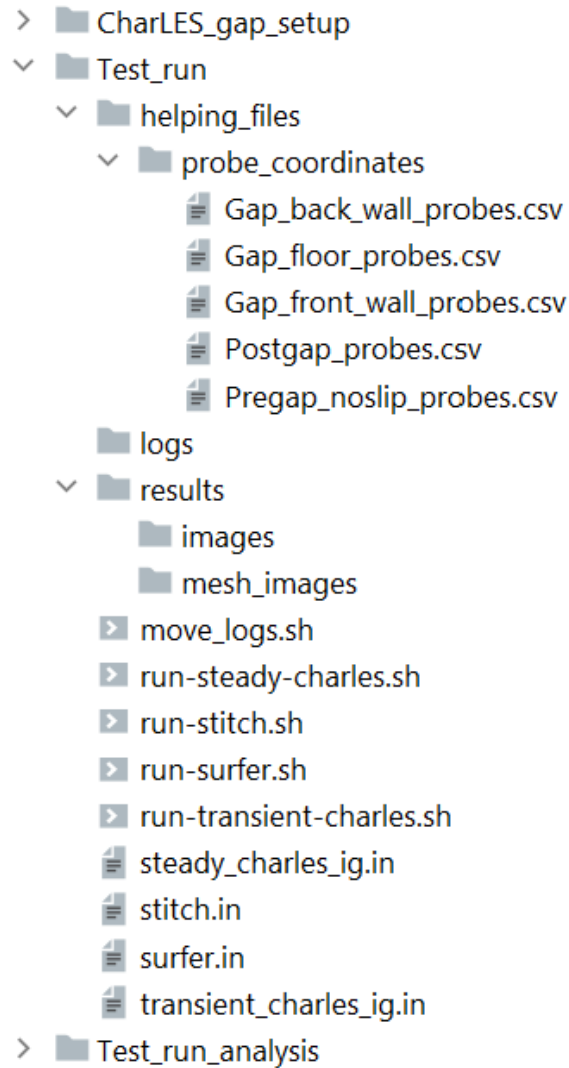


Figure 5: Simulation File Layout

generated files (the geometry and mesh files) will be created under './helper_files'. All your results should be saved in './results/' under appropriate subdirectories.

Note that all the probes are probeclouds (spacial probes) except for those placed along the shear layer which are simple probes (temporal probes). Also, the CharLES run is split into two parts - a transient part which doesn't save images or probe data, and a steady part which saves both and is meant for easy changes and reruns. Start by running the transient part, and only after that run the steady part.

2 Parameter Explanation

Most of the parameters used in the simulations are shown using Figures 6 and 7. Other parameters should be clear from their description in the input file. Notice that for NLAYERS you can give a list of numbers, where the first matches the layer closest to the surface, the next the layer after that, and so on. The probes also take multiple inputs - the number of probes wanted along the surface, the number of probes wanted in the direction perpendicular to the surface (should be 1 for a single line of probes), and the maximal distance the probes should reach from the surface (they always start on the surface, and 0 should be used if they are meant to all be on the surface).

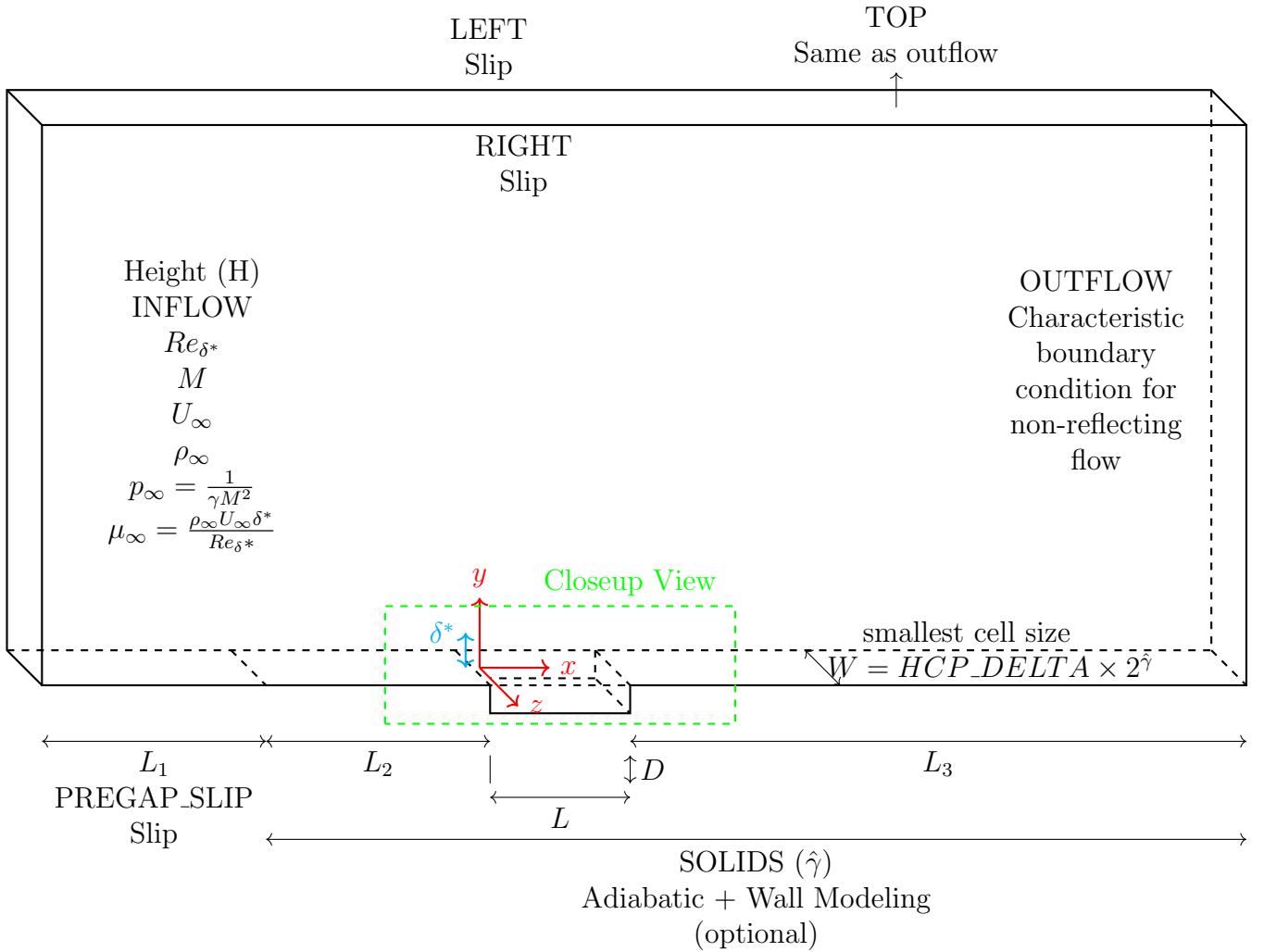


Figure 6: Simulation Setup With Parameters - Geometry

IMPORTANT NOTE: When generating files for a new simulation, make sure to change the simulation name (at the start of the 'input_parameters.txt' file). If a directory with the same name already exists where the new one is meant to be created, it will NOT be overridden (rather, the Python code will stop without generating any files). But be careful - if such a directory already exists on **zeus**, the python code will still create a new one on the local computer, and when you copy to zeus you will overwrite everything in

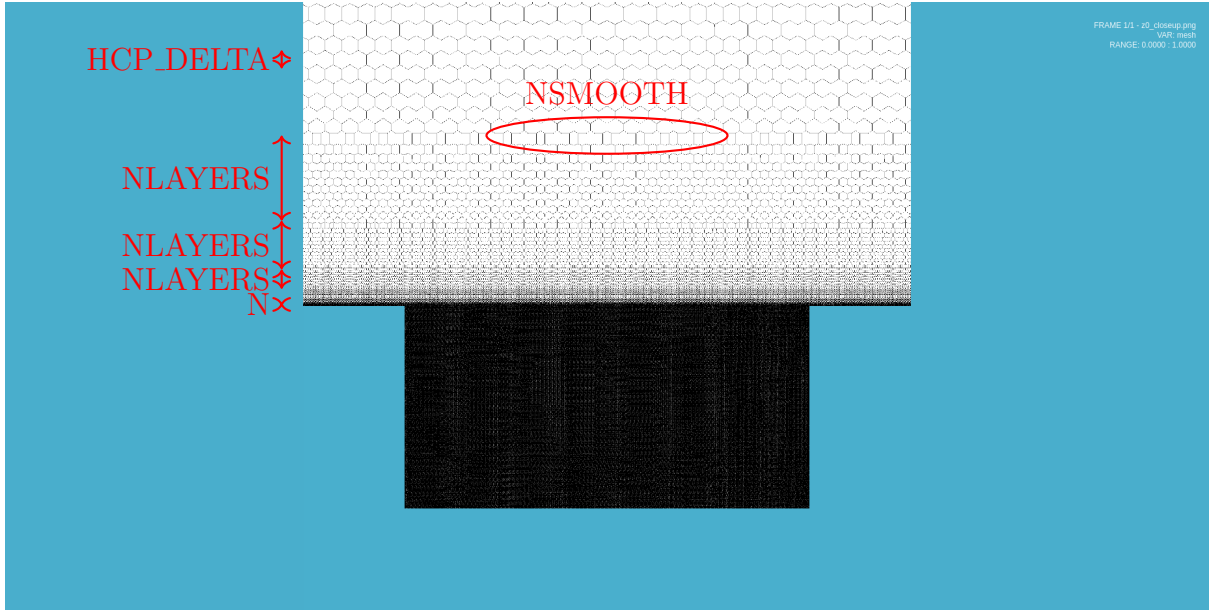


Figure 7: Simulation Setup With Parameters - Mesh

the directory on zeus. So using new simulation names is important.

In my own simulations, I have found that I was unable to create very fine meshes, getting the error from CharLES 'delta too small'. In an attempt to get around this, I introduce the parameter 'Scaling factor'. When my Python code runs, after calculating all the necessary parameters, it takes all the domain sizes and multiplies them by the scaling factor (thus changing the computational scale of the domain). This means that all the relations between the sizes are kept (including ' HCP_DELTA/δ^* '), but the actual sizes are increased, in order to allow CharLES to create smaller meshes. THIS HAS NOT BEEN TESTED. Proper testing would involve A. Checking that this indeed allows CharLES to create more refined meshes. B. Checking that a scaled version gives the same results as a non-scaled version (in the case that CharLES can create the mesh in a non-scaled version). -j 25.11.25 I beleive that this is unnecesary and that the problem was caused from trying to create too small meshes. For now this function still exists but is not expected to be used.

The information that the simulation is set up to save at the moment is images of the mesh in the stitch run, and in the steady CharLES run of the horizontal speed u , the vertical speed v , the dilation field $\nabla \cdot u$, the pressure field p , the density field ρ , the Mach field M , the temperature field T , and the vorticity field, in both the full and close-up views. There are also probes set up that match Figure 4. The probes save the values of u , v , p , ρ , T , $curl(u)$. There are also files written with data on y^+ along the surfaces.

Note that when you run the steady CharLES it resets the time in the domain. This is what allows it to run for the additional time set instead of stopping earlier because the simulation has reached the given time.

3 Debugging

In this section I'll write out some problems that I've had with running CharLES, and what I've done that seems to solve them.

1. If $\rho_{\text{dump}} = \text{nan}$ that means that my mesh is too fine (cells are too small).